

Lab 8

Objective: Based on section 11.4. This lab is to help you to

- Understand and implement a **copy constructor** for deep copying.
- Overload the **assignment operator** to handle assignment correctly, including self-assignment.
- Implement a **destructor** to release dynamically allocated memory or perform cleanup tasks.

Class Specifications:

- Private members:
 - `int* arr;` (pointer to dynamic array)
 - `int size;` (size of the array)
- Public methods:
 - Default constructor: Initializes the array to size 0.
 - Parameterized constructor: Accepts an integer to allocate an array of that size.
 - Destructor for the `DynamicArray` class to release the allocated memory when the object goes out of scope
 - Copy constructor to create a deep copy of a `DynamicArray` object, the copy constructor allocates new memory and copies the values from the source object.
 - Overload the assignment operator (`=`) to handle the assignment of one `DynamicArray` object to another.
 - Friend function of overloading operator `<<` to output the object information.

Finish the sample given below which practices the concepts listed above:

```
1  #include <iostream>
2  using namespace std;
3
4  class DynamicArray {
5  private:
6      int* arr;
7      int size;
8
9  public:
10     // Default constructor
11     DynamicArray() : arr(nullptr), size(0) {}
12
13     // Parameterized constructor
14     DynamicArray(int size) : size(size) {
15         arr = new int[size]();
16         for (int i = 0; i < size; ++i) {
17             arr[i] = i + 1; // Initialize array with some values
18         }
19     }
20 }
```

```

21 // Destructor
22 ~DynamicArray() {
23     delete[] arr;
24     cout << "Destructor called" << endl;
25 }
26
27 // Copy constructor
28 DynamicArray(const DynamicArray& other) {
29     size = other.size;
30     arr = new int[size];
31     for (int i = 0; i < size; ++i) {
32         arr[i] = other.arr[i];
33     }
34 }
35

```

```

36 // Overload assignment operator
37 DynamicArray& operator=(const DynamicArray& other) {
38     if (this == &other) {
39         return *this; // Handle self-assignment
40     }
41     delete[] arr; // Free existing memory
42     size = other.size;
43     arr = new int[size];
44     for (int i = 0; i < size; ++i) {
45         arr[i] = other.arr[i];
46     }
47     return *this;
48 }
49
50 friend ostream& operator <<(ostream& outs, const DynamicArray& theArray);
51 };
52

```

```

53 ostream& operator <<(ostream& outs, const DynamicArray& theArray)
54 {
55     outs<<"Items in the Array: "<<theArray.size<<endl;
56     for (int i = 0; i < theArray.size; ++i) {
57         outs << theArray.arr[i] << " ";
58     }
59     outs<<"\n";
60     return outs;
61 }
62 int main() {
63     int choice;
64     DynamicArray obj1, obj2;
65
66     int size;
67     cout << "Enter size of array: ";
68     cin >> size;
69     obj1 = DynamicArray(size); //use Parameterized constructor
70     cout<<obj1;
71
72     obj2 = obj1; // Copy using assignment operator
73     cout<<obj2;
74
75     DynamicArray obj3(obj1); // Copy using copy constructor
76     cout<<obj3;
77
78     return 0;
79 }

```

Submission:

You need to upload to Moodle: the source code (.cpp files), and the screenshot of the running of your program before 4pm Wednesday.